

SRE FOR GENEVA

A BEGINNER'S PROJECT GUIDE

Build, operate, break and recover a small service
using Linux, Python, C++, Docker, Prometheus and Grafana.
Designed for an in-person IP4IT / Geneva Business News lab.

YOUR STARTING POINT

CS MSc and distributed-systems foundation
Python / Linux / networking / full-stack experience to validate
Mac lab ready now; supervised physical-host step next

SIMON PARRIS | PRACTICAL EDITION | 19 SEPTEMBER 2026

Local project evidence, not a claim of production or CERN experience

0

How to use this book

Read Chapters 1-5 today, run the lab, then use one chapter per training session at IP4IT.

This book is written for your actual starting point: a completed computer-science master's degree, a distributed-systems foundation and user-reported exposure to Python, Linux, networking and full-stack development. Those are assets, but an employer needs observable proof that you can operate a service, not only build one.

The project is public at github.com/parrisi01/edge-sre-hardware-lab. It runs locally on a Mac today. At IP4IT, the goal is to replace the simulated device with an approved physical Linux host and have another person reproduce your runbook.

Today	Chapters 1-5 and the five-minute demo	You can explain each box and recover the injected fault
At IP4IT	Chapters 6-17 with a supervisor	Evidence is independently observed and recorded
Applications	Chapters 18-19	Every CV claim points to a file, test or witnessed exercise

Truth rule

The current system is a local simulation. Do not call it production, a data-centre deployment, FPGA work or CERN experience. The physical-host exercise becomes true only after it is completed and verified.

1

What SRE actually is

SRE is software engineering applied to operations: define reliability, measure it, automate repeatable work and learn from failure.

A developer asks whether the feature works. An operator asks whether the service stays useful over time. An SRE combines both views. You write code, but you also decide what 'healthy' means, observe the system, plan for failure and reduce manual work.

Shared ownership	The repository contains service code, deployment and runbook together
Automation	One command starts the complete stack; CI repeats tests
Measurement	Prometheus records polls, failures, availability and telemetry
Learning from failure	A safe device fault is injected and recovered
Reduce toil	Scripts replace repeated manual checks

The Google SRE material organizes the field around risk, SLOs, toil, monitoring, automation, release engineering, troubleshooting, incident response and postmortems. This book narrows those ideas to a small system you can fully understand.

Exercise

Someone says, 'SRE is just system administration with a new name.' What is missing?

Worked answer

System administration is part of the work, but SRE also uses software engineering, measurable service objectives, automated testing and repeatable incident learning. The distinguishing evidence is not the title; it is how the service is defined, measured and improved.



Read the employment evidence correctly

The labour-market case is Swiss and cross-industry; it is not evidence of a Geneva-specific SRE hiring boom.

Current Geneva evidence

Geneva FTE employment, Q2 2026	+1.0% quarter-on-quarter	Overall seasonally adjusted growth; excludes specified activities
Geneva FTE employment, 2024	+0.8% to 363,071	Broad structural series across the canton
Geneva secondary sector, 2024	+0.8%	Positive, but not a measure of IT occupations
Geneva information/communication, 2024	-1.7%	Contradicts a simple local IT-boom claim

OCSTAT reports economic branches, not the Site Reliability Engineer occupation. No authoritative Geneva SRE growth percentage was found. Therefore, this guide does not invent one.

Swiss ICT outlook

The 2025 ICT-Berufsbildung Schweiz study starts from about 266,000 ICT workers and projects 61,600 additional ICT positions through 2033 from economic growth and structural change. That is approximately 23.2% of the 2024 base across the forecast period, not an annual rate. It also projects 67,000 replacement needs, producing 128,600 gross personnel needs. The largest occupational components are software developers (46,200) and systems analysts (17,600).

More than two-thirds of Swiss ICT specialists work outside the core ICT industry. This supports portable reliability skills for finance, education, public administration, life sciences, logistics and industry. It does not guarantee a vacancy, interview, permit or junior-level hire.

Exercise

Can you write 'Geneva SRE jobs increased 23.2%' on a proposal?

Worked answer

No. The 23.2% is a calculation from a national forecast of additional ICT positions through 2033 divided by the 2024 Swiss ICT workforce base. It is neither Geneva-specific, SRE-specific nor annual. Use the exact scope and period.

2

Your existing hardware and the one optional purchase

Start with what is already connected. Only buy a physical host if IP4IT cannot lend one.

The project is deliberately usable before buying hardware. Begin with a Mac that can run Docker-compatible containers; add an independent Linux host only after the centre confirms the device and network.

Mac	Docker-compatible Compose, Git, make, Python and CMake	Runs the simulated lab and acts as operator workstation
External drive	Optional; approved and encrypted if it stores anything sensitive	Non-sensitive backup and restore drills; not a server
Ethernet adapter/cable	Optional until the physical-host phase	Connect an approved host on an approved lab network
Physical Linux host	Borrowed mini-PC, old laptop or Raspberry Pi	Independent boot, network, systemd service and hardware evidence

Storage rule

A public repository is not a backup of private data. Keep credentials, CVs, personal records and real organizational logs out of this project. If an external drive is used for sensitive material, verify its encryption and recovery method separately.

Optional purchase

The best-value complete device is the Raspberry Pi 5 4 GB Starter Kit: Pi 5, active-cooled case, official 27 W power supply and 32 GB microSD. Pi-Shop listed it at CHF 146.90 with immediate dispatch. Four gigabytes is enough for this lab. A local Digitec 8 GB board was CHF 179 and required separate accessories; pickup in Geneva and Zurich was shown for Monday, 21 September, not same-day.

Exercise

Can the external SSD replace a Raspberry Pi for the physical-host phase?

Worked answer

No. Storage cannot run its own operating system, network stack or service after the Mac is unavailable. Use the SSD for backup and restore evidence. Use a borrowed old laptop, mini-PC or Raspberry Pi as the independent Linux host.

3

The system you will operate

Understand the path from hardware-like telemetry to an operator decision.

Creates readings and faults	Carries one-line commands and responses	Polls, validates, stores and reports	Scrapes time-series metrics	Shows state and recovery evidence

The C++ process is intentionally small. It accepts READ, PING, SET_FAIL ON and SET_FAIL OFF. The Python service polls every two seconds. If telemetry is valid, it stores a sample and increments success. If the device reports a fault or cannot be reached, readiness fails but the control API remains alive so an operator can diagnose it.

Why separate liveness and readiness?

Liveness answers, 'Is the process running?' Readiness answers, 'Can it currently serve its intended function?' Restarting every unhealthy dependency can create loops. In this lab, the API remains alive during a device fault while readiness accurately reports that the service is not ready.

Exercise

During a simulated device fault, should the web API disappear completely?

Worked answer

No. The operator still needs status, metrics and recovery controls. The API stays live, while /readyz reports failure until healthy device telemetry resumes.

4

Run it today on the Mac

By the end of this chapter you will have a working dashboard and a recorded fault-recovery result.

Open Terminal and run:

```
git clone https://github.com/parrsi01/edge-sre-hardware-lab.git
cd edge-sre-hardware-lab
make setup
make up
open http://localhost:8000
```

Then run the scripted drill:

```
make demo
```

Expected sequence

Healthy	connected=true; telemetry is updating
Fault injected	device_status=fault; readiness fails; liveness stays available
Recovery	new successful sample appears; readiness returns
Evidence	availability reflects the failed poll instead of hiding it

The verified run on 19 September 2026 passed four Python tests, the C++ self-test, API/Prometheus/Grafana checks and the recovery drill. Availability became 95.652% because 22 of 23 polls were successful. That short-window value is evidence of the test, not a production reliability claim.

Exercise

Why did availability fall sharply after one brief failure?

Worked answer

The sample window was tiny: one failed poll out of 23 is about 4.35% failure. Over a long window, one failed poll would have less effect. Always state the numerator, denominator and time window.

5

Learn the repository

Be able to point to each component instead of saying that AI built it.

device-simulator/src/main.cpp	Protocol, telemetry and fault state	How is a client command parsed and answered?
control-api/app/main.py	Polling, state, routes and metrics	What happens when a poll fails?
control-api/tests/	Protocol tests	Which invalid inputs are rejected?
docker-compose.yml	Service graph and health checks	Which service waits for which dependency?
observability/	Prometheus and Grafana	Where does the dashboard get data?
hardware-agent/	Physical Linux replacement	Which values are real and which may be null?
scripts/	Operator workflow	How do you verify readiness before demoing?

Study method

For every file: read it, explain it aloud, predict one failure, then confirm the behavior. Change nothing until you can reproduce the baseline. An interview demonstration is strongest when you can diagnose a new fault without relying on a memorized script.

Exercise

What is the weakest honest answer to 'Did you build this yourself with AI?'

Worked answer

Do not hide AI assistance. Say: 'I used AI as an implementation assistant, then I ran the builds and tests, fixed packaging and readiness issues, and I can explain and operate every component. Here is the verification evidence and the next independently supervised step.'



Use the public GitHub repository

The public repository synchronizes project code, tests, guides and evidence boundaries without exposing the private Personal OS.

Public repository

Open github.com/parrisi01/edge-sre-hardware-lab. The README is the front door; TRAINING_GUIDE.md describes the work; SKILLS_MATRIX.md distinguishes current proof from future supervised proof; SECURITY.md defines the authorized boundary; and docs/ powers the public case-study site.

Get a fresh copy	<code>git clone https://github.com/parrisi01/edge-sre-hardware-lab.git</code>	The README and all source directories are present
See local changes	<code>git status</code>	Only intended files are listed
Inspect a change	<code>git diff</code>	No secret, personal path or unsupported claim
Record a reviewed change	<code>git add FILE; git commit</code>	Commit states the evidence-producing outcome
Synchronize	<code>git push</code>	GitHub shows the same commit and CI result

Private material does not belong here

Do not synchronize passwords, tokens, Gmail data, CVs, permit records, private Personal OS reports, internal IP addresses or centre logs. 'All tools and skills' means every tool and skill artifact for this lab—not the entire Mac or private account state.

Exercise

A file is useful for your job search but contains a home address. Should it be added?

Worked answer

No. Extract only the non-sensitive technical evidence needed for this project. Keep personal documents in private storage and link only to the public project.

6

Metrics without confusion

Use counters for accumulated events and gauges for current state.

Prometheus stores time series: numeric samples over time, identified by metric names and labels. The official guidance recommends measuring attempts, failures and latency for online services, and warns that unbounded labels create too many time series.

Counter	Only increases, except reset on restart	poll attempts and successful polls
Gauge	Current value can rise or fall	device connected state or temperature
Histogram	Counts observations in buckets	future poll-latency distribution
Summary	Client-side quantiles/count/sum	not needed for this first lab

Useful queries

```
edge_sre_device_connected
edge_sre_polls_total
edge_sre_poll_success_total
100 * edge_sre_poll_success_total / edge_sre_polls_total
```

A real SRE would normally calculate rates over a defined window and handle process restarts. The lab's direct ratio is deliberately readable for a first demonstration.

Exercise

Temperature changes from 39 C to 42 C and back. Counter or gauge? What about total failed polls?

Worked answer

Temperature is a gauge because it moves in both directions. Total failed polls is a counter because each failure adds one; derive a failure rate over a time window when alerting.

7

SLIs, SLOs, SLAs and error budgets

Define what users care about before choosing alerts.

SLI	Measured indicator	successful polls divided by total polls
SLO	Internal reliability target	99% successful polls during the agreed exercise window
SLA	External contractual promise with consequences	none; this training lab has no SLA
Error budget	Permitted unreliability under the SLO	1% of polls may fail under a 99% SLO

An SLO is only useful when its user, indicator, target and window are explicit. '99% reliable' is incomplete. 'At least 99% of scheduled device polls succeed during each 30-minute supervised drill' is measurable.

Availability arithmetic

At a 99% SLO over 10,000 polls, the error budget is 100 failed polls. At 99.9%, it is 10. Do not mix poll availability with user-request availability without explaining the difference.

Exercise

A 30-day service has a 99% availability target. What simple time budget does 1% represent?

Worked answer

Thirty days is 43,200 minutes. One percent is 432 minutes, or 7 hours 12 minutes. This is a simplified time-based budget; request-based SLOs require request counts instead.

8

Incident response

Respond in a predictable order: observe, contain, diagnose, recover, verify and learn.

Detect	Dashboard and readiness show the device fault
Declare	State start time, impact and owner aloud
Contain	Do not make unrelated changes; preserve logs
Diagnose	Check device status, API logs and container health
Recover	Disable the controlled fault or restart only the failed component
Verify	Wait for a new sample and readiness; check metrics
Learn	Write cause, timeline, recovery and one prevention action

Runbook command sequence

```
make status
make logs
curl -i http://localhost:8000/healthz
curl -i http://localhost:8000/readyz
make demo
```

Do not randomly restart everything first. That can erase evidence and may restore service without teaching you why it failed.

Exercise

The API is live, readiness fails and the device container is healthy. What do you check next?

Worked answer

Read the API status and logs for a protocol-level fault. A healthy process can still return an application fault. In this lab, check `device_status` and `last_error`, then use the documented recovery control.

9

Troubleshooting from layers

Find the lowest failing layer before changing anything.

Power/host	Is the machine up?	LEDs, console, uptime
Link	Is Ethernet physically connected?	link state and interface status
Network	Does it have an address and route?	ip address, route, ping where allowed
Transport	Is the TCP port listening?	ss/netstat and connection attempt
Application	Does the protocol return valid data?	READ/PING response and logs
Service	Can the user objective be completed?	readiness and fresh telemetry

Minimal diagnostic discipline

Record the exact symptom and timestamp. Reproduce once safely. Compare healthy versus failing evidence. Change one variable. Verify the user-visible result. Document the command and result. Escalate when authorization, safety or scope is unclear.

Exercise

A dashboard is blank. Name three different possible layers.

Worked answer

Browser/UI: JavaScript failed. API: the status endpoint is unavailable. Device: no fresh telemetry exists. Start from the browser request, check API health, then move down to the device; do not assume the first visible symptom identifies the root cause.

10

Containers and reproducibility

Containers make the lab repeatable; they do not remove the need to understand Linux and networking.

Docker Compose declares four services, their images, environment, health checks, ports, volumes and startup relationships. Colima provides the Linux virtual-machine runtime on this Mac. The host binds user-facing ports to 127.0.0.1, so the lab is not exposed to the local network by default.

device	none	none
control-api	healthy device	127.0.0.1:8000
prometheus	healthy control API	127.0.0.1:9090
grafana	Prometheus	127.0.0.1:3000

Build and verification

```
docker compose config --quiet
docker compose build
docker compose run --rm --no-deps control-api pytest -q
docker compose run --rm --no-deps device --self-test
```

Exercise

Why bind dashboards to 127.0.0.1 instead of 0.0.0.0 for the local demo?

Worked answer

127.0.0.1 limits access to the Mac itself. This fails closed while authentication is intentionally absent. Network access should be added only with an approved threat model and access control.

11

Toil and automation

Automate work that is manual, repetitive, predictable and safe to encode.

Toil is not simply 'work you dislike.' In SRE, it is operational work that is manual, repetitive, automatable, tactical and grows with the service. The setup, readiness and demo scripts remove repeated keystrokes while keeping the operator's decision visible.

Check every dependency	wait-ready.sh	whether the observed state is acceptable
Run builds and tests	verify.sh and CI	whether a failure blocks release
Inject and recover fault	demo.sh	whether this exercise is safe and authorized
Start all services	Docker Compose	configuration and exposure policy

Good automation is idempotent where practical, returns a meaningful exit code, emits useful errors and fails closed. A green script is evidence only if it actually tests the required outcome.

Exercise

Should a script automatically restart a failed service forever?

Worked answer

Not by default. Limited restart policies can improve availability, but infinite restarts can hide a persistent defect and consume resources. Add bounded retries, observability and an escalation path.

12

Move from simulator to a real Linux host

The physical step is a protocol replacement, not a rewrite of the whole system.

First ask IP4IT for an approved Linux mini-PC, old laptop, Raspberry Pi or workstation on an isolated lab network. If none is available, the Raspberry Pi 5 4 GB kit is the recommended purchase. Do not connect an unmanaged personal device to a production or office network without approval.

Physical sequence

1	Install supported Linux and record version	OS and kernel output
2	Create a restricted service account	account and file ownership
3	Enable key-based SSH on the approved network	successful login and disabled unnecessary access
4	Install hardware-agent/edge_agent.py as a systemd service	enabled service and logs
5	Point the control API at the host	fresh real-host samples
6	Disconnect link or stop service under supervision	detected incident and measured recovery
7	Have another participant follow the runbook	handover result and corrections

The agent reports real Linux thermal data only when the kernel exposes it. Unavailable values remain null. Never fabricate voltage or sensor readings.

Exercise

What converts this from a software demo into hardware-aware operational evidence?

Worked answer

A separately powered Linux host, real network link, supervised service deployment, a physical or process failure, measured recovery and an independent witness. Owning a Raspberry Pi alone is not evidence.

13

Use the external SSD correctly

Turn the existing drive into recovery evidence without treating it as a server.

An approved external drive can hold a versioned, non-sensitive project backup. Exclude secrets and personal data. The safest first exercise backs up only this public repository.

Backup drill - review the destination before running

```
backup_root=${EDGE_SRE_BACKUP_ROOT:?Set EDGE_SRE_BACKUP_ROOT to an approved destination}
stamp=$(date +%Y%m%d-%H%M%S)
dest="$backup_root/$stamp"
mkdir -p "$dest"
rsync -a --exclude '.git' --exclude '__pycache__' ./ "$dest/"
find "$dest" -maxdepth 2 -type f | sort
```

Restore verification

Do not restore over the live project. Restore into a new temporary directory, compare checksums or file lists, run the tests from the restored copy, then delete the temporary copy only after review.

```
restore_dir=$(mktemp -d)
rsync -a "$dest/" "$restore_dir/"
cd "$restore_dir"
docker compose config --quiet
echo "Restored copy: $restore_dir"
```

Exercise

A backup command returned exit code 0. Is recovery proven?

Worked answer

No. A backup is only proven after a restore to a separate location and validation of the required files or service. Record the restore result and time.

14

Security and safe lab boundaries

Reliability includes preventing your own tools from creating a new incident.

Network exposure	host ports on 127.0.0.1	approved isolated network and documented firewall
Privileges	containers run non-root where defined	restricted service user; no shared admin account
Secrets	none required	do not commit passwords, tokens or Wi-Fi credentials
Data	synthetic telemetry	collect only approved host metrics
Faults	simulated command	supervised and reversible; never on production
Publishing	public code uses synthetic data	centre approves every centre name, image or internal detail

Never packet-capture other people's traffic. Wireshark exercises at IP4IT must use your own lab traffic or an explicitly provided capture. Never scan networks or hosts without written authorization.

Exercise

Can you publish a screenshot showing the centre's internal IP addresses?

Worked answer

Not without explicit approval. Use synthetic addresses or crop/redact internal identifiers. Public evidence should prove your method without exposing the centre.

15

Six-week IP4IT execution plan

Each week ends with observable evidence and a supervisor decision.

1	Baseline, architecture, Linux/network assessment	Rebuild and explain the Mac lab without prompts
2	Deploy agent to approved physical Linux host	systemd service survives reboot and emits real available metrics
3	Run three controlled failures	symptom, diagnosis, action and recovery recorded
4	Security and configuration	restricted account, approved exposure, no secrets in repo
5	SLO and recovery	window defined; backup and restore drill passes
6	Independent handover and interview	another participant runs the guide; reviewer scores evidence

Daily 60-minute pattern

0-10	State the objective and expected evidence
10-35	Perform one lab change or drill
35-45	Run tests and inspect metrics/logs
45-55	Write the result and one remaining gap
55-60	Commit locally or prepare review; choose next action

Ask IP4IT to classify every skill as performed independently, performed with help, or not yet demonstrated. A factual record is more valuable than a vague certificate.

16

Your first IP4IT meeting

Show a working system first, then ask for a narrow supervised physical extension.

Opening script

"I have prepared a small hardware-aware reliability lab. It polls a C++ device, validates telemetry in Python, exposes metrics, detects a controlled fault and recovers. The local version works. I would like to replace the simulator with one approved Linux host here, practise network and service failures, and produce a runbook another participant can follow."

Five-minute demonstration

0:00	Architecture	One sentence per component
0:45	Healthy dashboard	State current SLI and its small window
1:30	Inject fault	Explain liveness versus readiness
2:30	Logs and Grafana	Point to the causal evidence
3:30	Recover	Verify a fresh sample, not only a green light
4:15	Repository/tests	State limits and physical next step

Questions to obtain decisions

- Which approved Linux device and isolated network may I use?
- Who can review one hour per week and witness the final drill?
- Which existing tools - GLPI, OCS, Nagios or Wireshark - should the project integrate with, if any?
- What data and screenshots may be used in a public portfolio?
- What participation, funding and work-authorization framework applies?

Exercise

What is the meeting's definition of done?

Worked answer

A named supervisor, one approved host, an allowed network boundary, a six-week schedule, a decision on public evidence, and a clear answer on participation/funding/authorization. Interest without these decisions is not a project start.

17

Connect to IP4IT's actual programme

Use the centre's published tools only where they serve the project.

The Geneva Business News IT page describes daily practice across support, development, systems and networking, a weekly IT session, technical presentations, labs, and work with GLPI, OCS, Nagios and Wireshark. That supports the project's direction, but it does not prove that a place, supervisor, device, funding or specific installation is available.

Support	runbook, incident ticket and handover	access to real user tickets
Development	Python API, C++ device and tests	production deployment
Systems	Linux service, logs, backup and restore	administrator privileges
Networking	TCP protocol, link failure and diagnosis	permission to inspect the office network
Tools	possible ticket, inventory, monitoring or packet-analysis integration	that every named tool is currently installed

Best integration sequence

First make the physical host work. Second create one GLPI ticket for a controlled incident if their GLPI is available. Third compare the lab's Prometheus alert with their approved Nagios process. Fourth inspect only your own lab TCP exchange in Wireshark. OCS integration is optional and should follow the centre's actual inventory architecture.

Exercise

Why not install all four named tools immediately?

Worked answer

Tools are not the objective. Reliability evidence is. Installing unused software adds complexity and security risk. Integrate the smallest centre-approved tool that improves a real workflow.

18

Turn the project into employment evidence

Apply for roles using verified outcomes, not a promise that the project qualifies you for everything.

Best-aligned role families

Junior SRE / platform	health checks, metrics, incident drill, automation	production/on-call experience
DevOps / cloud	containers, CI, repeatable deployment	cloud/IaC at organizational scale
Linux / infrastructure	systemd host, network diagnosis, restore	enterprise fleet and identity tooling
Control software	C++, TCP, Python, hardware boundary	real device/DAQ/FPGA domain
Application support	runbook, logs, incident reproduction	specific business application knowledge

Truthful CV bullet after physical completion

Built and operated a four-service hardware-aware reliability lab using C++, Python/FastAPI, Docker Compose, Prometheus and Grafana; deployed a telemetry agent to an approved Linux host, documented three controlled incidents and validated recovery through an independent runbook exercise.

Until the physical stage is complete, remove the phrase 'deployed ... to an approved Linux host' and keep the work under Projects, not Employment.

CERN and other employer references

Hardware-aware software roles often combine Python, modern C++, device interaction, REST/web interfaces and CI/CD. This lab addresses those technical themes, but it cannot replace a vacancy's formal experience, nationality, degree or domain requirements. Verify every live vacancy independently before applying.

Exercise

Where should this appear on your CV today?

Worked answer

Under Projects as 'Edge SRE Hardware Lab - local simulation.' Link to the reviewed repository only after publication. Do not list IP4IT as employment unless a real employment relationship exists.

19

Assessment and next action

Finish with proof that you can operate and explain the system without hiding its limits.

Architecture	Draw all five components and explain each boundary
Operation	Start, inspect and stop the stack from the guide
Failure	Detect, diagnose and recover a surprise fault
Measurement	Calculate an SLI with real numerator, denominator and window
Linux	Run agent as a restricted systemd service on approved hardware
Networking	Explain link, IP, TCP and application layers using evidence
Recovery	Restore a separate copy and validate it
Communication	Deliver the five-minute demo and truthful limits
Handover	Another person succeeds using your runbook

Today's exact action

1. Open the dashboard. 2. Run `make demo`. 3. Read Chapters 1-5. 4. Explain the architecture aloud without reading. 5. Write one paragraph describing what failed and how recovery was verified.

Definition of done: you can reproduce the demo, state the difference between liveness and readiness, identify the numerator and denominator of availability, and name the one physical-host decision IP4IT must make.

No purchase is required today. Ask for a loaned approved host first. If none is available, recheck the Raspberry Pi 5 4 GB kit's total price and delivery immediately before purchasing. No order has been made.

A

Command reference

Use this page during the demonstration; understand every command before an interview.

Project operations

```
git clone https://github.com/parrsi01/edge-sre-hardware-lab.git
cd edge-sre-hardware-lab
make setup # ensure the container runtime is running
make up # build/start services
make status # list container health
make logs # follow service logs
make demo # fault and recovery exercise
./scripts/verify.sh
make down
```

HTTP checks

```
curl -i http://localhost:8000/healthz
curl -i http://localhost:8000/readyz
curl -s http://localhost:8000/api/status | jq
curl -s http://localhost:8000/metrics | head
```

Views

```
open http://localhost:8000
open http://localhost:8000/docs
open http://localhost:9090
open http://localhost:3000/d/edge-sre-lab
```

Physical Linux checks

```
hostnamectl
ip address
ip route
ss -lntp
systemctl status edge-agent
journalctl -u edge-agent --since '10 minutes ago'
```

Commands shown for the physical host are exercises for an authorized lab system. Never run network inspection or service changes on systems you do not own or have permission to administer.

B

Sources and evidence notes

Primary sources support the concepts; all project-specific results were separately tested locally.

Core learning sources

- Google, Site Reliability Engineering - Table of Contents: risk, SLOs, toil, monitoring, automation, troubleshooting, incidents and postmortems.
- Google, The Site Reliability Workbook: practical SLO, monitoring, incident and configuration chapters.
- Prometheus, Instrumentation best practices: attempts, errors, latency, metric types and label-cardinality cautions.
- Raspberry Pi, Getting started and remote access: boot media, network and SSH setup.
- Geneva Business News, IT programme page: support/dev/system/network practice, labs and named tools.
- OCSTAT, Geneva employment Q2 2026 and detailed 2024 results.
- ICT-Berufsbildung Schweiz, ICT workforce requirements through 2033: national cross-industry forecast, not a Geneva vacancy count.

Procurement sources checked 19 September 2026

- Pi-Shop Raspberry Pi 5 4 GB Starter Kit.
- Digitec Raspberry Pi 5 8 GB board and live collection panel.
- Digitec Geneva store and Digitec locations.

Evidence boundaries

The official IP4IT/GBN pages establish programme alignment only. Hardware, placement, supervision, funding and work authorization remain unknown until the centre confirms them. Store prices and stock are volatile. The user's skills and education are user-reported; the project build and demo results were observed locally. Publication proves access to code and documentation, not independent operation or employment.

Repository records

```
README.md
TRAINING_GUIDE.md
SKILLS_MATRIX.md
SECURITY.md
https://github.com/parrsi01/edge-sre-hardware-lab
```